

Mini-projet INFO2 Algorithmique Avancée

Yahia Ben Letaief, Hocine Kerkoub

3 avril 2026



Résumé

Compte rendu sur le mini projet d'Algorithmique Avancée, où l'on étudie la triangulation d'un polygone convexe composé de n sommets, ici en langage Python.

Table des matières

Introduction	3
A. Questions préliminaires	4
1. Nombre de cordes distinctes dans un polygone à n sommets	4
2. Toutes les triangulations d'un polygone contiennent $n - 3$ cordes	4
B. Essais successifs	6
1. Fonction Valide corde	6
2. Étude d'un algorithme	7
a. Calcul de plusieurs fois la même triangulation	7
b. Limite de la méthode	7
3. Construire toute triangulation une fois et une seule	8
a. Calcul de plusieurs fois la même triangulation	8
b. Complexité de l'algorithme	8
c. Condition d'élagage	10
d. Test de la fonction optimale	10
C. Programmation dynamique	11
1. Formule de calcul d'une triangulation minimale d'un polygone convexe de n côtés	11
2. Algorithme programmation dynamique	11
3. Complexité	11
4. Intérêt de la décomposition et modifications possibles	12
D. Algorithme glouton	13
1. Mise en oeuvre de la méthode	13
2. Caractère exact de la stratégie gloutonne	14
E. Recommandation Argumentée	15
F. Annexes	16

Introduction

La triangulation d'un polygone est un problème classique et fondamental en géométrie algorithmique, trouvant des applications majeures dans des domaines variés tels que la modélisation 3D, la conception graphique ou encore les systèmes d'information géographique. Dans le cadre de ce mini-projet d'Algorithmique Avancée, nous nous concentrons sur un problème d'optimisation spécifique : la triangulation minimale d'un polygone convexe. L'objectif est de décomposer un polygone à n sommets en un ensemble de triangles, en traçant des cordes qui ne se croisent pas, de telle sorte que la somme des longueurs de ces cordes soit la plus petite possible.

Ce rapport détaille notre démarche pour aborder, modéliser et résoudre ce problème en comparant différents paradigmes de programmation. Dans un premier temps, nous poserons les bases théoriques du problème à travers des questions préliminaires, permettant notamment de quantifier le nombre de cordes nécessaires pour toute triangulation valide.

Ensuite, nous mettrons en œuvre et analyserons trois approches algorithmiques distinctes :

- La méthode des essais successifs (Backtracking) : une recherche exhaustive qui garantit l'obtention d'une solution optimale, mais dont nous étudierons les limites sévères liées à l'explosion combinatoire.
- La programmation dynamique : une approche optimale s'appuyant sur la résolution de sous-problèmes imbriqués, permettant de réduire considérablement la complexité temporelle.
- L'algorithme glouton : une stratégie heuristique faisant systématiquement le choix localement optimal (la corde la plus courte), dont nous analyserons les performances et démontrerons la non-exactitude globale.

Enfin, nous concluons par une recommandation argumentée, appuyée sur des tests de performances chronométrés et une analyse poussée des complexités spatiales et temporelles de chaque méthode, afin de déterminer l'algorithme le plus adapté à la résolution de ce problème.

A. Questions préliminaires

1. Nombre de cordes distinctes dans un polygone à n sommets

On considère un polygone convexe à n sommets.

Étape 1 : compter tous les segments possibles

On peut former un segment entre chaque paire de sommets distincts. Le nombre total de paires de sommets est :

$$\binom{n}{2} = \frac{n(n-1)}{2}.$$

Cela correspond à tous les segments possibles (arêtes et cordes).

Étape 2 : retirer les arêtes du polygone

Parmi ces segments, certains correspondent aux arêtes du polygone, c'est-à-dire aux segments reliant deux sommets consécutifs. Un polygone à n sommets possède exactement n arêtes.

Ces segments ne sont pas des cordes, il faut donc les exclure.

Étape 3 : calcul du nombre de cordes

Les cordes sont les segments reliant deux sommets non adjacents. On obtient donc :

$$\text{Nombre de cordes} = \frac{n(n-1)}{2} - n.$$

En simplifiant :

$$\frac{n(n-1) - 2n}{2} = \frac{n(n-3)}{2}.$$

Conclusion : Un polygone à n sommets possède exactement

$$\boxed{\frac{n(n-3)}{2}}$$

cordes distinctes.

2. Toutes les triangulations d'un polygone contiennent $n - 3$ cordes

Montrons que toute triangulation d'un polygone à n sommets comporte le même nombre de cordes.

Pour cela, pour $n \geq 3$, notons $\mathbf{HR}_n$ l'hypothèse : Toute triangulation d'un polygone à n sommets convexe comporte exactement $n - 3$ cordes.

Cas de base : $n = 3$, Polygone à 3 sommets (Triangle) :

On obtient alors de façon évidente que, pour tout triangle, la seule triangulation possible est composé de 0 cordes, d'où $\mathbf{HR}_3$ est vérifiée.

Cas général : $n \geq 3$, Polygone à n sommets :

Supposons $n \geq 3$ tel que $\mathbf{HR}_n$ est vérifié. Montrons $\mathbf{HR}_{n+1}$;

Soit P un polygone convexe composé des sommets $(i_0, i_1, \dots, i_n)$.

Soit $j \in \llbracket 1, n+1 \rrbracket$, notons P' le polygone composé des sommets $(i_0, \dots, i_{j-1}, i_{j+1}, i_n)$, c'est à dire P privé du sommet i_j .

Alors P' est un n -polygone convexe, et par $\mathbf{HR}_n$, P' est triangularisable en $n - 3$ cordes.

Or le triangle i_{j-1}, i_j, i_{j+1} est constitué de 2 arêtes de P et d'une arête de P' .

Ainsi, en ajoutant ce triangle à P' , on obtient P et une triangulation composée de $n - 3 + 1 = (n + 1) - 3$ cordes.

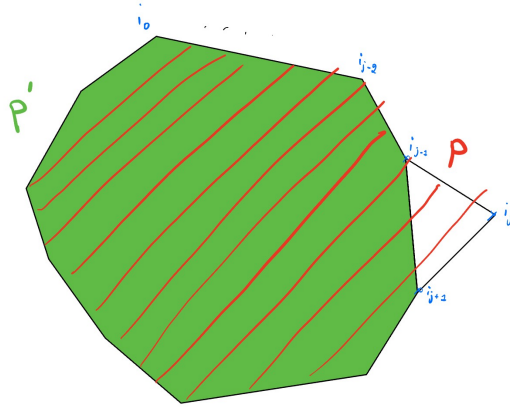


FIGURE 1 – Exemple d'un couple de polygone (P, P')

Conclusion :

Ceci étant vrai pour tout $j \in \llbracket 1, n+1 \rrbracket$, on en déduit que toute triangulation de P contient $(n+1) - 3$ cordes. D'où $\mathbf{HR}_{n+1}$.

Ainsi, nous avons démontré que *toute triangulation d'un polygone à n sommet comporte le même nombre de cordes.*

B. Essais successifs

Dans cette section, nous nous intéressons à la recherche de la solution optimale du problème de triangulation minimale d'un polygone convexe en utilisant la méthode des essais successifs (backtracking). Cette approche consiste à explorer toutes les combinaisons possibles de cordes et de sélectionner celles qui produisent une triangulation valide avec la longueur totale minimale.

1. Fonction Valide corde

Le but de cette fonction est de vérifier si une corde entre 2 sommets est valide ou non. Comme indiqué dans l'énoncé on suppose que les cordes sont contenues dans la matrices T où $T[i][j] = \text{True}$ si la corde existe est déjà tracé False sinon.

Algorithm 1: validecorde

```
Data: Deux indices  $i, j \geq 0$ 
Result:  $\text{True}$  si la corde  $(i, j)$  est valide,  $\text{False}$  sinon
/* Vérifie si la corde  $(i, j)$  peut être ajoutée sans croiser d'autres cordes
   existantes */
if  $i > j$  then
  | échanger  $i$  et  $j$  // On s'assure que  $i > j$  pour simplifier les comparaisons
end
if  $i = j$  then
  | return  $\text{False}$  // Une corde ne peut pas relier un sommet à lui-même
end
if  $|i - j| = 1$  ou  $|i - j| = n - 1$  then
  | tooling max. 2 requests /sec  $\text{False}$  // Les cordes interdite entre sommets
  | consécutifs, suivis ou extrémités du polygone
end
if  $T[i][j] = \text{True}$  then
  | return  $\text{False}$  // La corde existe déjà
end
/* Vérification des croisements avec toutes les cordes existantes */
for  $k \leftarrow 0$  to  $n - 1$  do
  | for  $l \leftarrow k + 1$  to  $n - 1$  do
    | if  $T[k][l] = \text{True}$  then
      | if  $k > l$  then
        | | échanger  $k$  et  $l$  // On force toujours  $k < l$  pour simplifier les
        | | comparaisons et opérations
      | end
      | if  $k \in \{i, j\}$  ou  $l \in \{i, j\}$  then
        | | continuer // Ignorer les cordes qui touchent  $i$  ou  $j$ 
      | end
      | if  $i < k < j$  et  $j < l$  then
        | | return  $\text{False}$  // La corde  $(i, j)$  croise la corde  $(k, l)$ 
      | end
      | if  $k < i < l$  et  $l < j$  then
        | | return  $\text{False}$  // Autre cas de croisement
      | end
    | end
  | end
end
return  $\text{True}$  // Corde  $(i, j)$  valide si aucune condition n'a été violée
```

2. Étude d'un algorithme

a. Calcul de plusieurs fois la même triangulation

Oui, il est possible que l'algorithme calcule plusieurs fois la même triangulation. En effet, une triangulation peut être obtenue par différents ordres de choix des cordes. Par exemple, si une triangulation comporte les cordes (s_0, s_2) et (s_0, s_3) , on peut d'abord choisir (s_0, s_2) puis (s_0, s_3) , ou inversement. Chacun de ces chemins dans l'arbre des choix conduit à la même triangulation finale, ce qui entraîne des répétitions.

b. Limite de la méthode

Contre-exemple :

Considérons un polygone convexe à $n = 6$ sommets (hexagone) notés $s_0, s_1, s_2, s_3, s_4, s_5$ dans l'ordre.

Triangulation considérée : On considère la triangulation définie par les cordes :

$$(s_2, s_4), \quad (s_2, s_5), \quad (s_0, s_2).$$

Cette triangulation est valide (aucune intersection) et comporte bien $n - 3 = 3$ cordes.

Dans la triangulation considérée, la corde (s_0, s_2) doit être tracée. Cependant, cette corde doit être choisie à l'étape $i = 0$.

Supposons que l'algorithme choisisse de ne rien tracer à l'étape 0.

Alors :

- La corde (s_0, s_2) ne pourra plus être ajoutée ultérieurement, car l'algorithme ne considère les cordes issues de s_0 qu'à l'étape 0.
- Il devient donc impossible de construire cette triangulation.

Réciproquement, si l'algorithme choisit à l'étape 0 une autre corde issue de s_0 , par exemple (s_0, s_3) , alors :

- Cette corde peut interdire certaines cordes nécessaires à la triangulation (en créant des intersections).
- Là encore, la triangulation considérée ne pourra pas être obtenue.

Conclusion :

Cette stratégie impose un ordre strict de décision : chaque corde doit être choisie au moment précis où son sommet de départ est traité. Si ce choix n'est pas effectué à ce moment-là, il devient impossible par la suite.

Ainsi, certaines triangulations valides ne peuvent jamais être générées par cet algorithme.

3. Construire toute triangulation une fois et une seule

a. Calcul de plusieurs fois la même triangulation

Pour cet algorithme on utilise la fonction récursive backtracking qui s'occupera de faire la recherche récursive des solutions et dès qu'une solution est trouvée, si cette solution est meilleure que la meilleure solution trouvée, on remplace les valeurs de la meilleure solution et du meilleur poids.

Algorithm 2: triangulation_minimale

Data: Liste des cordes C contenant (a, b, ℓ) , nombre de sommets n

Result: Une triangulation minimale et son poids

meilleur_poids $\leftarrow +\infty$

meilleure_solution $\leftarrow []$

Fonction backtracking(i , $solution$, $poids_courant$) :

if $|solution| = n - 3$ **then**

if $poids_courant < meilleur_poids$ **then**

 meilleur_poids $\leftarrow poids_courant$

 meilleure_solution $\leftarrow copie(solution)$

end

return

end

if $i = |C|$ **then**

return

end

 /* Cas 1 : ne pas prendre la corde $C[i]$ */

*/

 backtracking($i + 1$, $solution$, $poids_courant$)

 /* Cas 2 : essayer de prendre la corde $C[i]$ */

*/

$(a, b, \ell) \leftarrow C[i]$

if validecorde(a, b) **then**

$T[a][b] \leftarrow \text{True}$

$T[b][a] \leftarrow \text{True}$

 ajouter $C[i]$ à solution

 backtracking($i + 1$, $solution$, $poids_courant + \ell$)

 retirer le dernier élément de solution

$T[a][b] \leftarrow \text{False}$

$T[b][a] \leftarrow \text{False}$

end

backtracking(0, $[]$, 0)

return meilleure_solution, meilleur_poids

b. Complexité de l'algorithme

Pour étudier la complexité en nombre d'appels récursifs il faut surtout se concentrer sur la fonction backtracking. Dans un polygone à n sommets, le nombre de cordes possibles est de l'ordre de $O(n^2)$.

À chaque étape, l'algorithme décide pour chaque corde de l'inclure ou non dans la solution. Il y a donc deux choix possibles pour chaque corde.

Ainsi, l'arbre de récursion est binaire et de profondeur $O(n^2)$, ce qui donne un nombre total de nœuds de l'ordre de :

$$O(2^{n^2})$$

Coût d'un appel : À chaque appel récursif, la fonction validecorde est utilisée pour vérifier si une corde peut être ajoutée sans intersection. Cette vérification nécessite de parcourir les cordes déjà sélectionnées, ce qui prend un temps en $O(n^2)$ dans le pire cas.

Complexité totale :

En combinant le nombre d'appels récursifs et le coût de chaque appel, on obtient :

$$O(n^2 \cdot 2^{n^2})$$

c. Condition d'élagage

On choisit comme condition d'élagage le fait que si le poids de la solution en cours de recherche est supérieur ou égale au poids de la meilleure solution on arrête la recherche car il n'est alors pas possible que cette solution soit meilleure que l'actuelle.

Par contre la complexité

$$poids_courant \geq meilleur_poids$$

On choisit cette condition d'élagage pour la version optimisée de notre algorithme, même si la complexité temporelle en pire cas ne change pas.

Algorithm 3: triangulation_minimale_opt

Data: Liste des cordes C contenant (a, b, ℓ) , nombre de sommets n

Result: Une triangulation minimale et son poids

$meilleur_poids \leftarrow +\infty$

$meilleure_solution \leftarrow []$

Fonction `backtracking`($i, solution, poids_courant$) :

```

    if  $|solution| = n - 3$  then
        if  $poids\_courant < meilleur\_poids$  then
            meilleur_poids  $\leftarrow$  poids_courant
            meilleure_solution  $\leftarrow$  copie(solution)
        end
        return
    end
    if  $i = |C|$  then
        return
    end
    /* Élagage (branch and bound) */
    if  $poids\_courant \geq meilleur\_poids$  then
        return
    end
    /* Cas 1 : ne pas prendre la corde  $C[i]$  */
    backtracking( $i + 1, solution, poids\_courant$ )
    /* Cas 2 : essayer de prendre la corde  $C[i]$  */
     $(a, b, \ell) \leftarrow C[i]$ 
    if validecorde( $a, b$ ) then
         $T[a][b] \leftarrow \text{True}$ 
         $T[b][a] \leftarrow \text{True}$ 
        ajouter  $C[i]$  à solution
        backtracking( $i + 1, solution, poids\_courant + \ell$ )
        retirer le dernier élément de solution
         $T[a][b] \leftarrow \text{False}$ 
         $T[b][a] \leftarrow \text{False}$ 
    end
backtracking(0, [], 0)
return meilleure_solution, meilleur_poids

```

d. Test de la fonction optimale

On observe que pour $n = 13$ l'algorithme optimisé avec élagage s'exécute en 16,10s, cette durée en exécutant notre programme en ajoutant la commande `time` ou `Measure-Command` dans le terminal mais pour $n = 14$ on a une durée de 2m35s avec la même méthode. Donc le dernier nombre de sommet utilisable en moins de 2 minutes est $n = 13$

C. Programmation dynamique

1. Formule de calcul d'une triangulation minimale d'un polygone convexe de n côtés

Formule de récurrence pour la triangulation minimale

On note $T_{i,t}$ le coût minimal de triangulation du sous-polygone formé par les sommets $s_i, s_{i+1}, \dots, s_{i+t-1}$.

Cas de base :

Si le sous-polygone contient moins de trois sommets, aucune triangulation n'est nécessaire.

Ainsi :

$$T_{i,t} = 0 \quad \text{si } t < 3.$$

Cas général :

Pour $t \geq 3$, on choisit un sommet intermédiaire s_{i+k} avec $1 \leq k \leq t-2$ pour former un triangle $(s_i, s_{i+k}, s_{i+t-1})$.

Ce choix découpe le problème en deux sous-problèmes indépendants :

- le sous-polygone $T_{i,k+1}$,
- le sous-polygone $T_{i+k,t-k}$.

Le coût associé à ce choix correspond à la somme des longueurs des cordes ajoutées pour former le triangle, à savoir :

$$d(s_i, s_{i+k}) + d(s_{i+k}, s_{i+t-1}) + d(s_i, s_{i+t-1}).$$

Ce qui vaut dans notre cas

$$d(s_i, s_{i+k}) + d(s_{i+k}, s_{i+t-1}).$$

car s_i, s_{i+t-1} est une arête du polygone.

On obtient alors la formule suivante :

$$T_{i,t} = \min_{1 \leq k \leq t-2} \left(T_{i,k+1} + T_{i+k,t-k} + d(s_i, s_{i+k}) + d(s_{i+k}, s_{i+t-1}) \right).$$

2. Algorithme programmation dynamique

On implémente l'algorithme basé sur la formules de récurrence précédente :

3. Complexité

Complexité spatiale : L'algorithme utilise une matrice T de taille $n \times n$ pour stocker les solutions des sous-problèmes $T_{i,t}$, où n est le nombre de sommets du polygone. Ainsi, la complexité spatiale est :

$$\mathcal{O}(n^2)$$

Complexité temporelle : Pour remplir la matrice, l'algorithme parcourt :

- une boucle sur la longueur des sous-polygones allant de 3 à n ,
- une boucle sur l'indice de départ i des sous-polygones,
- une boucle sur le sommet intermédiaire k choisi pour former un triangle.

Chaque boucle peut s'exécuter au plus n fois dans le pire des cas. Ainsi, le nombre d'opérations élémentaires est proportionnel à :

$$\sum_{t=3}^n \sum_{i=0}^{n-t} \sum_{k=i+1}^{i+t-2} 1 \approx \mathcal{O}(n^3)$$

Algorithm 4: progDynamique

```
Data: Liste des sommets points
Result: Poids minimal de la triangulation
 $n \leftarrow |points|$ 
 $T \leftarrow$  matrice  $n \times n$  initialisée à 0
/* On considère des sous-polygones de taille croissante */
for  $longueur \leftarrow 3$  to  $n$  do
    /* Choix du sommet de départ */
    for  $i \leftarrow 0$  to  $n - longueur$  do
         $j \leftarrow i + longueur - 1$ 
         $T[i][j] \leftarrow +\infty$ 
        /* On teste toutes les décompositions possibles */
        for  $k \leftarrow i + 1$  to  $j - 1$  do
             $cout \leftarrow 0$ 
            /* Ajout de la corde  $(i, k)$  si elle n'est pas sur le bord */
            if  $k \neq i + 1$  then
                 $cout \leftarrow cout + distance(points[i], points[k])$ 
            end
            /* Ajout de la corde  $(k, j)$  si elle n'est pas sur le bord */
            if  $k \neq j - 1$  then
                 $cout \leftarrow cout + distance(points[k], points[j])$ 
            end
            /* Mise à jour du minimum */
             $T[i][j] \leftarrow \min(T[i][j], T[i][k] + T[k][j] + cout)$ 
        end
    end
end
/* Résultat pour le polygone complet */
return  $T[0][n - 1]$ 
```

4. Intérêt de la décomposition et modifications possibles

Cette manière de décomposer le problème est intéressante car elle garantit que les cordes choisies pour la triangulation ne se croisent pas, assurant ainsi la validité de la solution. En effet, chaque sous-problème partage toujours un sommet avec ses sous-problèmes voisins, ce qui permet de construire progressivement le polygone triangulé tout en réutilisant les résultats intermédiaires de manière efficace. Si l'on procédait en sélectionnant deux cordes quelconques à chaque étape, il serait nécessaire de vérifier explicitement que ces cordes ne se croisent pas entre elles ni avec les cordes déjà tracées. Cette vérification supplémentaire compliquerait l'algorithme et augmenterait le nombre de sous-problèmes à considérer, ce qui pourrait accroître significativement la complexité temporelle. L'approche avec un sommet commun simplifie donc à la fois la gestion des sous-problèmes et la construction des triangulations minimales.

D. Algorithme glouton

1. Mise en oeuvre de la méthode

La particularité de cet algorithme est qu'il est dit "**glouton**", ainsi, il effectue à chaque instant le meilleur choix possible sur le moment, sans retour en arrière ni anticipation des étapes suivantes, dans l'objectif d'atteindre au final un résultat optimal.

De plus, on suppose l'existence de la matrice booléenne T , comme dans la partie B, indiquant si une corde est présente ou non.

Enfin, on utilise la fonction `validecorde(i, j)` définie dans la partie B, qui vérifie si l'ajout de la corde (i, j) ne crée pas d'intersection avec les cordes déjà présentes.

Algorithm 5: algoGlouton

```
Data: Liste des sommets sommets
Result: Une triangulation
solution  $\leftarrow []$ 
 $n \leftarrow |sommets|$ 
 $c \leftarrow 0$ 
/* Initialisation de la matrice des distances */
 $C \leftarrow$  matrice  $n \times n$ 
for  $i \leftarrow 0$  to  $n - 1$  do
    for  $j \leftarrow 0$  to  $n - 1$  do
         $C[i][j] \leftarrow \text{distance}(sommets[i], sommets[j])$ 
    end
end
/* On ajoute des cordes jusqu'à obtenir une triangulation ( $n - 3$  cordes) */
while  $c < n - 3$  do
     $m \leftarrow +\infty$ 
     $meilleur \leftarrow (-1, -1, -1)$ 
    /* Recherche de la meilleure corde (la plus courte) */
    for  $i \leftarrow 0$  to  $n - 1$  do
        for  $j \leftarrow 0$  to  $n - 1$  do
            /* On ignore les arêtes du polygone */
            if  $i \notin \{(j - 1) \bmod n, j, (j + 1) \bmod n\}$  then
                /* On vérifie que la corde n'est pas déjà choisie */
                if  $(i, j, C[i][j]) \notin solution$  et  $(j, i, C[j][i]) \notin solution$  then
                    /* On choisit la plus petite corde valide */
                    if  $C[i][j] < m$  et  $\text{validecorde}(i, j)$  then
                         $m \leftarrow C[i][j]$ 
                         $meilleur \leftarrow (i, j, m)$ 
                    end
                end
            end
        end
    end
     $(a, b, \_) \leftarrow meilleur$ 
    /* Ajout de la corde */
     $T[a][b] \leftarrow \text{True}$ 
     $T[b][a] \leftarrow \text{True}$ 
    solution.ajouter( $meilleur$ )
     $c \leftarrow c + 1$ 
end
return solution
```

Pour étudier la complexité de l'algorithme glouton, il faut se concentrer sur le comportement des différentes boucles. Chacune des boucles `while` et `for` sont répétées n fois. De plus, au niveau de la condition `if`, la vérification que $(i, j, T[i][j]) \notin solution$ nécessite de parcourir et de comparer chacun des éléments de solutions avec le triplet, d'où une complexité en $O(n)$

Ainsi, en prenant en compte la complexité temporelle de chacune des boucles et des conditions, on obtient que la complexité temporelle de l'algorithme glouton est en :

$$O(n^4)$$

On remarque également que la complexité spatiale de l'algorithme est celle de C , une matrice de dimension $n \cdot n$.

Ainsi, la complexité spatiale de l'algorithme glouton est en :

$$O(n^2)$$

2. Caractère exact de la stratégie gloutonne

Nous pouvons remarquer que la stratégie gloutonne n'est pas exacte. En effet, l'algorithme se précipite sur la corde la plus courte, c'est à dire l'optimum local. Le problème est que cet optimum local peut nuire à l'obtention d'un optimum global, puisque parfois en prenant une combinaison de corde de taille moyenne, nous obtenons une combinaison qui serait plus efficace qu'une combinaison avec une corde plus courte, qui aurait été choisi par l'algorithme glouton.

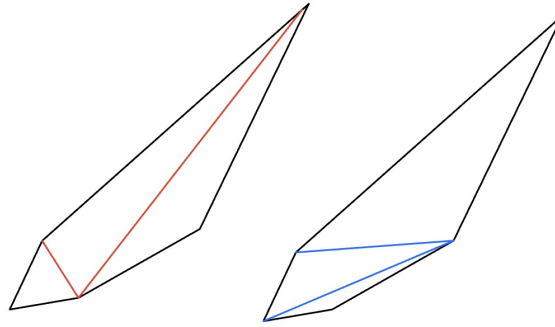


FIGURE 2 – Schéma montrant pour un polygone pour $n = 5$

- **En rouge :** Algo Glouton. Ici l'algorithme glouton choisit la corde la plus courte, ce qui le force derrière à devoir choisir la corde la plus longue. Le poids des cordes ici vaut :
 $p_{glouton} = p_{c1} + p_{c4} = 1 + 5.8 = 6.8$
- **En Bleu :** Prog Dynamique (optimale). Ici l'algorithme optimal choisit les deux cordes de taille moyenne. Le poids des cordes ici vaut :
 $p_{optimal} = p_{c2} + p_{c3} = 2.5 + 3.3 = 5.8$

On remarque qu'ici, l'ensemble $\mathcal{C}_{tot}$ des cordes de ce polygone de taille $n = 5$ correspond à :
 $\mathcal{C}_{glouton} \cup \mathcal{C}_{optimal}$

Ainsi, l'ensemble des combinaisons de cordes a été considéré, ce qui montre la non-exactitude de l'algorithme glouton.

E. Recommandation Argumentée

Il y a trois éléments à prendre en compte pour la résolution de ce problème : l'exactitude de la solution obtenue, ainsi que les complexités temporelle et spatiale des algorithmes.

Nous avons démontré précédemment que l'algorithme glouton ne fournit pas toujours une solution optimale, contrairement aux algorithmes de programmation dynamique et de backtracking, qui garantissent l'obtention d'une triangulation minimale.

Du point de vue des performances, l'algorithme le plus efficace en termes de complexité temporelle est celui de programmation dynamique, avec une complexité en $O(n^3)$ et une complexité spatiale en $O(n^2)$, ce qui reste raisonnable pour des tailles d'entrée modérées.

En revanche, l'algorithme de backtracking, bien qu'exact, présente une complexité temporelle exponentielle, de l'ordre de $O(n^2 \cdot 2^n)$, ce qui le rend impraticable pour des valeurs de n élevées.

Enfin, des tests expérimentaux ont été réalisés afin d'observer les temps d'exécution des différents algorithmes. Les résultats obtenus sont présentés ci-dessous et permettent de montrer les différences de performance entre les approches.

n	essais successifs	essais successifs élagués	programmation Dynamique	Glouton
11	1.01s	0,66s	0.03s	0.03s
12	6,09s	3,55s	0.03s	0.03s
13	39,88s	16,10s	0,03s	0,03s
14	5m51	2m35	0.03s	0.03s
50	trop long	trop long	0.04s	0.24s
100	trop long	trop long	0.11s	5.2s

On remarque que même pour de petit valeur de n on voit une différence de temps d'exécution entre cette méthode et les 2 autres algorithmes. Alors que pour voir la différence entre l'algorithme de programmation dynamique et le glouton il faut monter a de grande valeur n . Mais il ne faut pas oublier que l'algorithme glouton ne donne pas toujours la solution optimale.

Conclusion

Pour conclure, l'algorithme recommandé pour résoudre ce problème est l'algorithme de programmation dynamique. En effet, il présente une complexité temporelle polynomiale, contrairement à l'approche par essais successifs qui est exponentielle. De plus, son coût mémoire reste raisonnable, et il garantit l'obtention d'une solution optimale.

Les prochaines étapes serait d'étudier la possibilité d'utiliser des tactiques DIVISER POUR MIEUX RÉGNER, de sorte à tenter de réduire la complexité et potentiellement s'approcher d'une complexité temporelle quadratique ou pseudo-linéaire.

Enfin il serait intéressant d'étendre le problème à tout polygone, convexe ou concave. En effet, comme tout polygone concave peut être divisé comme adjonction de plusieurs polygones convexes, la tactique précédente du DIVISER POUR MIEUX RÉGNER semble pouvoir s'appliquer, de sorte à réduire la taille du problème à plusieurs sous problèmes à une échelle logarithmique.

F. Annexes

Code Python : Triangulation de polygone

```
# =====
# Generation de polygone de test
# =====

import math
import random

def generer_polygone(n, rayon_min=50, rayon_max=150, seed=42):
    random.seed(seed)
    angles = sorted([random.uniform(0, 2 * math.pi) for _ in range(n)],
                    reverse=True)
    sommets = []
    for angle in angles:
        rayon = random.uniform(rayon_min, rayon_max)
        x = rayon * math.cos(angle)
        y = rayon * math.sin(angle)
        sommets.append((x, y))
    return sommets

def distance(p1, p2):
    return math.dist(p1, p2)

def generer_vecteur_C(sommets):
    n = len(sommets)
    cordes = []
    for i in range(n):
        for j in range(i + 2, n):
            if i == 0 and j == n - 1:
                continue
            longueur = distance(sommets[i], sommets[j])
            cordes.append((i, j, longueur))
    return cordes

# =====
# Initialisation
# =====

n = 13
sommets = generer_polygone(n)
cordes = generer_vecteur_C(sommets)
T = [[False] * n for _ in range(n)]
```



```

# =====
# Fonction validecorde
# =====

def validecorde(i, j):
    if i > j:
        i, j = j, i
    if i == j:
        return False
    if abs(i - j) == 1 or abs(i - j) == n - 1:
        return False
    if T[i][j]:
        return False
    for k in range(n):
        for l in range(k + 1, n):
            if T[k][l]:
                if k > l:
                    k, l = l, k
                if k in (i, j) or l in (i, j):
                    continue
                if i < k < j and j < l:
                    return False
                if k < i < l and l < j:
                    return False
    return True

```

```

# =====
# Backtracking
# =====

def triangulation_minimale(cordes, n):
    meilleur_poids = float("inf")
    meilleure_solution = []

    def backtracking(i, solution, poids_courant):
        nonlocal meilleur_poids, meilleure_solution

        if len(solution) == n - 3:
            if poids_courant < meilleur_poids:
                meilleur_poids = poids_courant
                meilleure_solution = solution.copy()
            return

        if i == len(cordes):
            return

        backtracking(i + 1, solution, poids_courant)

        (a, b, longueur) = cordes[i]
        if validecorde(a, b):
            T[a][b] = True
            T[b][a] = True
            solution.append(cordes[i])

            backtracking(i + 1, solution, poids_courant + longueur)

            solution.pop()
            T[a][b] = False
            T[b][a] = False

    backtracking(0, [], 0)
    return meilleure_solution, meilleur_poids

```

```

# =====
# Programmation dynamique
# =====

def progDynamique(points):
    n = len(points)
    T = [[0] * n for _ in range(n)]

    for longueur in range(3, n + 1):
        for i in range(n - longueur + 1):
            j = i + longueur - 1
            T[i][j] = float("inf")

            for k in range(i + 1, j):
                cout = 0
                if k != i + 1:
                    cout += distance(points[i], points[k])
                if k != j - 1:
                    cout += distance(points[k], points[j])

                T[i][j] = min(T[i][j], T[i][k] + T[k][j] + cout)

    return T[0][n - 1]

```

```

# =====
# Algorithme glouton
# =====

def algoGlouton(sommets):
    solution = []
    n = len(sommets)
    c = 0

    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            C[i][j] = distance(sommets[i], sommets[j])

    while c < n - 3:
        m = float("inf")
        meilleur = (-1, -1, -1)

        for i in range(n):
            for j in range(n):
                if i not in [(j - 1) % n, j, (j + 1) % n]:
                    if ((i, j, C[i][j]) not in solution and
                        (j, i, C[j][i]) not in solution):

                        if C[i][j] < m and validecorde(i, j):
                            m = C[i][j]
                            meilleur = (i, j, m)

        a, b, _ = meilleur
        T[a][b] = True
        T[b][a] = True
        solution.append(meilleur)
        c += 1

    return solution

```